

ICSC 2026

Universities HACKATHON

Challenge Statements & Problem Briefs

8

TRACKS

8

CHALLENGES

GUIDING PRINCIPLE

We define the problem. You decide the solution.

Each challenge in this document describes a real problem and explains why it matters. None of them tells you what to build. The method, the tools, the data and the part of the problem you choose to attack are entirely up to you.

HOW TO READ A CHALLENGE

Every challenge in this document has the same four parts.

01

The problem

What is going wrong, and why it matters.

02

Things to keep in mind

The real conditions your solution must survive.

03

What to build

What your team must show the judges.

04

Free tools to start with

So that no team is blocked by cost.

Before you choose a track, read the rules on the next page. They apply to all eight challenges.

Rules that apply to every track

Read this page before you choose a track.

01 No real personal data

Use synthetic, simulated or properly anonymised data only. Be ready to explain where your data came from and how you handled it.

02 Build something

A slide deck on its own is not a submission. We want a working prototype, model, analysis or demonstration.

03 A prototype is enough

You are not expected to deliver a finished commercial product. Scope your work to what you can finish and show.

04 Spend nothing

Every challenge here can be completed with free and open-source tools on a normal laptop. Paid tools earn no extra marks.

05 Explain your results honestly

Show where your solution fails, not only where it works. Judges will reward a team that knows its own limits.

06 Think about power and network cuts

In Nigeria these are normal, not rare. Say what happens to your solution when it is offline for a few hours. You do not have to solve it, but you should have thought about it.

THE EIGHT TRACKS

	DOMAIN	CHALLENGE
A	Financial Services & Digital Payments	Spotting Account Takeover from Behaviour
B	Digital Identity & Trust	Proving a Fact Without Revealing the Whole Record
C	Health & Medical Systems	Safe Access to Patient Records
D	Government & Public Sector	Sorting Incident Reports Nobody Has Time to Read
E	Critical Infrastructure & Energy	Catching Unsafe Commands in Your Own Control System
F	Education & Campus Systems	Protecting University Research
G	Commerce & Consumer Protection	Watching What Third Party Integrations Really Do
H	Media & Civic Trust	Proving Digital Evidence Has Not Been Changed

A

TRACK A

Financial Services & Digital Payments

Where most Nigerians meet cybercrime first: mobile money, USSD and agency banking.

A1. Spotting Account Takeover From Behaviour

THE PROBLEM

Most Nigerian bank accounts are protected by something a criminal can steal: a four-digit PIN, a password, or an OTP sent by SMS. Fraudsters phish them, buy them, or talk victims into reading the code out over the phone. Once the correct secret is entered, the system assumes the right person is there. It checks the secret, not the human being.

Banks already collect signals that could tell them more. How fast someone types. Whether a field was typed or pasted. How the phone is held. What time this customer normally sends money. What screens they usually visit first. Almost none of this is used. Meanwhile the rules that do exist are blunt, such as flagging any large or first-time transfer, so they block honest customers and miss patient criminals who stay just under the limit.

THINGS TO KEEP IN MIND

- The decision must happen in under a second, inside a live transfer.
- Many customers use USSD or feature phones, so there is no app collecting signals.
- Real fraud examples are rare, arrive late, and are heavily outnumbered by normal transactions.
- Blocking a genuine transfer is not harmless. It might be someone's rent or hospital bill.
- Families share phones and people change SIMs. Neither is fraud.

WHAT TO BUILD

- A synthetic dataset of user sessions and transactions, including normal users and attackers. Say clearly how you generated it.
- A model that scores a session or transaction as it happens.
- Proof that it catches takeovers without flooding the bank with false alarms. Show both numbers.
- A plain sentence explaining each decision, of the kind a bank could give a customer who asks why their transfer was stopped.
- A sensible answer for USSD and feature phone users, even if it is a different one.

FREE TOOLS YOU CAN START WITH

Python, scikit-learn, pandas. Public keystroke dynamics datasets exist for behavioural signals. A simple simulated banking app is enough to collect your own timing data.

B1. Proving a Fact Without Revealing the Whole Record

THE PROBLEM

Nigerians prove who they are many times a year: registering a SIM, opening an account, entering a building, collecting a benefit, claiming a student discount. Almost every time, they hand over far more than the question needed. A bar that only needs to know if you are over eighteen receives your full date of birth, your name, your address and your ID number.

The result is that full copies of identity records now sit in hundreds of places: an agent's spreadsheet, a school's shared drive, a small startup's database, a photocopy in a drawer. Each copy is a separate chance for a leak, and a leak exposes an entire identity at once. You cannot reissue your date of birth the way you reset a password. This stolen data then powers SIM swaps and account takeovers.

The assumption underneath is that checking an identity requires seeing the record. It does not. The person asking almost never needs the record. They need a trustworthy yes or no.

THINGS TO KEEP IN MIND

- The businesses doing the checking are small and not technical. If it is hard to use, they will not use it.
- Some citizens have basic phones, weak signal and limited digital confidence.
- The business still needs proof that it carried out a check, even if it keeps no data.
- A business can always just demand the full ID anyway. Think about why yours would be used instead.

WHAT TO BUILD

- A working demo where a common question, such as "is this person over 18" or "is this person verified", is answered without releasing the underlying record.
- A simple screen for the business doing the checking. Simple enough that a shop owner could use it.
- A record that proves the check happened, without storing the personal data.
- An explanation of what stops someone from faking a yes.

FREE TOOLS YOU CAN START WITH

You do not need advanced cryptography to make a strong entry, although you may use it. Open-source libraries for digital signatures, verifiable credentials and QR codes are freely available. A mock identity database that you create yourself is fine.

C1. Safe Access to Patient Records

THE PROBLEM

Nigerian hospitals are moving from paper files to computers quickly, and often without securing them. Access is usually all or nothing, so a records clerk, a visiting doctor and a departing intern can all open the same complete patient histories. Staff share logins because shifts change and getting a new account takes too long. Where audit logs exist, they sit on the same machine they are supposed to be watching, so anyone who breaks in can edit them.

Health records are among the most sensitive data there is. A leak can expose an HIV status, a pregnancy, a mental health history or a genotype, with real consequences for someone's job, marriage, family and safety.

There is a second danger. A hospital that cannot open its records cannot safely treat patients. Ransomware against hospitals is now common worldwide, and the pressure to pay is high because the alternative is measured in harm to patients. Any control you design has to survive a real ward, where a doctor in an emergency cannot be stopped by a login screen.

THINGS TO KEEP IN MIND

- Emergency access must always be possible. It is also an obvious way to abuse the system.
- Staff move between wards and shifts constantly and share devices.
- Most hospitals have very few IT staff, unstable power and weak internet.
- The records system was bought, not built, so you usually cannot replace it.

WHAT TO BUILD

- A prototype records system, or a layer on top of a simple one, where what a user can see depends on their role, their ward and their current duty.
- A working emergency override that grants access immediately and makes that access impossible to hide.
- An access log that stays trustworthy even if the main system is compromised.
- A demonstration of at least one abuse case being caught, such as a clerk opening records of patients they never treated.
- A short answer for what staff do when the system is down or offline.

FREE TOOLS YOU CAN START WITH

Any web framework you know, plus SQLite or PostgreSQL. Synthetic patient data generators such as Synthea are free. Hash chaining is enough to make a log tamper evident.

D1. Sorting Incident Reports Nobody Has Time to Read

THE PROBLEM

When something goes wrong in a Nigerian institution, the first report is rarely a neat security alert. It is a message. A staff member emails to say the payroll portal asked for a password twice. A school administrator forwards a WhatsApp message about a strange link. A CERT receives the same incident four times from four organisations, each describing it differently, none of them aware of the others.

Someone then has to read all of it and work out what type of incident it is, how serious it is, whether it is the same as another report, what technical details are buried in the text, and who needs to know. In most institutions that person does not exist or is one of two people already doing everything else. So, reports get handled in the order they arrived instead of the order that matters, and the account takeover happening right now waits behind fourteen phishing emails nobody opened.

The reports are also messy. They are written by people who are not security experts, in a mix of English and Pidgin, with screenshots pasted instead of text and the important detail in the last line. They often contain customer or patient details that should never be passed on.

THINGS TO KEEP IN MIND

- Reports are free text of very mixed quality. There is no standard form.
- The same incident arrives many times with no shared reference number and no shared vocabulary.
- The reporter usually does not know how serious it is.
- Sending a report to the wrong place means nobody acts on it.
- Reports often contain personal data that must be removed before sharing onward.

WHAT TO BUILD

- ▶ A set of at least a few hundred realistic incident reports that you create yourself, with a correct answer recorded for each one. Explain how you built it.
- ▶ A pipeline that reads a raw report and returns: incident type, how serious it is, any technical details found in the text, who it should go to, and a cleaned version that is safe to share.
- ▶ Grouping so that repeat reports of one incident are recognised as one incident.
- ▶ Results measured against your own answer key, including the cases you get wrong.
- ▶ A simple queue screen showing the most urgent reports first.

FREE TOOLS YOU CAN START WITH

Python with scikit-learn or spaCy is enough for the whole pipeline. You may run a small open model locally with Ollama if you want, but it is not required, and a simpler model

you can explain will score better than a prompt you cannot. Public incident categories such as MITRE ATT&CK give you a ready-made labelling scheme.

E

TRACK E

Critical Infrastructure & Energy

Where a cyber incident stops being digital and becomes physical.

E1. Catching Unsafe Commands in Your Own Control System

THE PROBLEM

The systems that open valves, start pumps and switch power feeders talk to each other using old industrial protocols such as Modbus and MQTT. These protocols check nothing. If a command is correctly formatted, it is obeyed. Such systems were once kept safe by being disconnected from everything else, but remote support links and office network connections have quietly removed that protection.

When someone abuses this, the damage is physical, not just digital. The hardest cases are the ones where nothing looks broken. The command is valid, the format is correct, the sender looks legitimate, but it is sent at the wrong moment or to equipment in a state where it causes harm. A pump switched on against a closed valve. A setpoint moved a little at a time until it leaves the safe range. An old reading replayed so the control room stays calm while a tank empties.

Real attack data does not exist, and no student will be given access to a power station. So build your own small system. A simulated water tank with a pump and a valve, speaking a real industrial protocol, reproduces every problem above. Because you built it, you can attack it and know the right answer.

THINGS TO KEEP IN MIND

- Assume the attacker can send perfectly valid, well formatted commands.
- Keeping the plant running matters more than keeping secrets. Blocking a genuine safety command can cause the accident you were trying to prevent.
- Normal traffic is very repetitive, so anything unusual stands out. The problem is that legitimate maintenance also looks unusual.
- The person reading your alerts is an engineer under pressure, not a security specialist.
- Your system must not shut the process down by itself. It advises. A human decides.

WHAT TO BUILD

- ▶ A working simulated process controlled over Modbus TCP or MQTT, showing normal running, start up, shut down and at least one legitimate maintenance activity.
- ▶ Your own attack scripts: injecting a command, replaying old traffic, sending a valid command at the wrong moment, and slowly drifting a setpoint. Keep the code so your data can be reproduced.
- ▶ A detector, tested on that data. Catching a malformed packet is the easy case. The real result is catching the valid command that should not have been sent.
- ▶ An alert screen that tells an engineer what happened, to which equipment, and why it matters, plus a clear statement of what your system does when it is unsure.

FREE TOOLS YOU CAN START WITH

Python, pymodbus or Eclipse Mosquitto, a tank model in a few dozen lines, and Wireshark. An ESP32 board is welcome but is **not required and earns no extra marks**. Spend most of your time on the detector, not the simulator.

F

TRACK F

Education & Campus Systems

The environment you know best; with problems nobody has fixed.

F1. Protecting University Research

THE PROBLEM

University laboratories hold work whose value depends on not being seen too early: unpublished results, draft patent applications, new methods, and datasets gathered over many years. If this leaks before it is published or filed, a patent can be lost, and years of work can be handed to a better funded competitor.

The protection around it is usually close to nothing. Files sit on a shared drive with permissions built up over years, on a supervisor's personal cloud account, on students' laptops, and in WhatsApp groups that never get deleted. Research is naturally collaborative and mobile, so files are forwarded, copied and carried on flash drives every day. Postgraduate students and research assistants arrive and leave every year, and nobody ever takes their access away.

The real difficulty is not encryption. It is people. Research culture depends on sharing, and any control heavy enough to work will simply be avoided by a researcher facing a deadline, who will email a copy to themselves and move on.

THINGS TO KEEP IN MIND

- Collaboration crosses departments, universities and countries. It cannot be blocked.
- Researchers under pressure will bypass anything that slows them down.
- Staff and students turn over every year, and nobody removes old access.
- Campus networks are deliberately open, and IT teams are stretched thin.
- Some of this material must be shared outside eventually, under clear terms.

WHAT TO BUILD

- A working prototype that makes sensitive research files harder to copy, forward or take away without permission, while still allowing normal collaborative work.
- Detection of at least a few realistic leak patterns, such as a mass download before a student leaves, or a file being sent to an outside address.
- A controlled way to share a file with an external partner, with a record of what was shared and with whom.
- Evidence that people would use it. Test it on a few real students and report what they did, including what annoyed them.

FREE TOOLS YOU CAN START WITH

Python or any web stack, plus free file storage such as Nextcloud if you want a realistic base. You can create your own synthetic research files and simulate a departing student to test detection.



TRACK G

Commerce & Consumer Protection

Your platform is made of other people's software. So is your risk.

G1. Watching What Third Party Integrations Really Do

THE PROBLEM

A modern Nigerian online store, delivery company or fintech is built out of other people's services. Payments, identity checks, delivery tracking, messaging, analytics and marketing all arrive as integrations, and each one is given access to customer data when it is connected. That access is usually far wider than needed, because setting narrow permissions is fiddly and the team was in a hurry to launch.

Then everybody forgets. Nobody checks what a supplier can still reach. An integration built for a two-week trial is still live three years later. Keys are copied between environments or left in code by a developer who has since resigned. The company's real exposure is everything all its integrations can reach, and almost none of them could produce that list if asked.

The failures follow naturally. A supplier is breached and the attacker inherits its access to your customers. An integration quietly pulls far more data than its job requires. All of this is hard to spot because the traffic is authorised, expected and encrypted, so it looks exactly like normal business until customer data turns up somewhere it should not.

THINGS TO KEEP IN MIND

- This traffic is allowed and looks completely normal. That is the whole difficulty.
- Real usage genuinely changes with seasons, sales campaigns and new partner features.
- Cutting off an integration by mistake can stop payments, deliveries or sign ups.
- The company carries the legal and reputational blame when a partner leaks data.

WHAT TO BUILD

- A simulated platform with several integrations, some behaving normally and some misbehaving.
- A live map of what data each integration can reach and what it touches.
- Detection of abnormal partner behaviour while it is happening, such as pulling far more records than usual or reaching a data type it never used before.
- A graded response rather than an on or off switch. Show what you would do at each level and why.
- Proof that a busy sales day does not trigger a false alarm.

FREE TOOLS YOU CAN START WITH

Any web framework, plus an API gateway you write yourself. Generating your own API logs is the easiest way to get labelled data.

A large, bold, white letter 'H' on a dark teal square background.

TRACK H

Media & Civic Trust

Evidence is only useful if you can prove nobody changed it.

H1. Proving Digital Evidence Has Not Been Changed

THE PROBLEM

Cybercrime cases, fraud investigations and disciplinary panels increasingly depend on digital evidence: server logs, transaction records, phone extractions, CCTV footage, message exports. That evidence is only useful if you can show it is genuine and unchanged since it was collected. In Nigerian investigations, that often cannot be shown.

Evidence gets copied onto a flash drive, emailed between officers, saved on a shared computer, and handed over as a folder of files. Whether the copy matches the original, who has touched it, and whether anything was changed are usually unanswerable questions. Handling records are on paper, if they exist at all. Cases are then lost on integrity grounds, not because anyone tampered with the evidence but because nobody can prove they did not.

There is one more requirement that is easy to forget. The final audience is a magistrate, a judge or a panel member who is not a technologist. A process that is technically perfect but cannot be explained in court has not solved the problem.

THINGS TO KEEP IN MIND

- Evidence is collected in the field, under pressure, often by officers with no forensic training.
- Files are large. One phone extraction can be over a hundred gigabytes.
- Cases run for years, across staff changes and equipment replacement.
- Evidence usually contains other people's personal data, which must be protected.
- The argument must convince someone with no technical background.

WHAT TO BUILD

- A working prototype that records evidence from the moment of collection and makes any later change detectable.
- A custody trail showing every person who handled the item and what they did.
- A demonstration where you deliberately alter a file and your system proves it.
- A simple report that a judge or panel member could read and understand, with no technical jargon.
- A sensible approach to collecting evidence with no network connection, syncing later.

FREE TOOLS YOU CAN START WITH

Hashing and hash chains are built into standard libraries in every language. You do not need a blockchain, and if you use one you should be able to explain why it was necessary. Any small set of sample files works as test evidence.